

Database Engineering for Developers & DBAs

Chapter 2: Database Design

একটি সফটওয়্যার বা তথ্যভিত্তিক (Data-driven) সিস্টেমের সফলতার অন্যতম প্রধান ভিত্তি হলো **সঠিক Database Design**। একটি ভালো Database Design শুধু ডেটা সংরক্ষণই করে না, বরং দ্রুত Query, উচ্চ Performance, নিরাপত্তা (Security), Scalability এবং ভবিষ্যতে সহজ Maintenance নিশ্চিত করে।

1. Database Design কী?

Database Design হলো এমন একটি পরিকল্পিত প্রক্রিয়া, যার মাধ্যমে একটি Database-এর Structure তৈরি করা হয়। এতে নির্ধারণ করা হয়—

- কী ধরনের Data সংরক্ষণ করা হবে।
- কোন কোন Table থাকবে।
- Table-গুলোর মধ্যে কী Relationship থাকবে।
- কোন Column Primary Key বা Foreign Key হবে।
- কীভাবে Data Efficient এবং Secureভাবে সংরক্ষণ করা হবে।

সহজভাবে বলতে গেলে,

Database Design হলো ব্যবসায়িক চাহিদা (Business Requirements) অনুযায়ী এমন একটি Database Structure তৈরি করা, যা দ্রুত, নির্ভরযোগ্য (Reliable) এবং ভবিষ্যতে সহজে সম্প্রসারণযোগ্য (Scalable)।

Database Design-এর উদ্দেশ্য

- Data Redundancy কমানো।
- Data Integrity নিশ্চিত করা।
- Performance বৃদ্ধি করা।
- Scalability নিশ্চিত করা।
- Database সহজে Maintain করা।
- Security নিশ্চিত করা।

Database Engineering for Developers & DBAs

বাস্তব উদাহরণ

ধরুন, একটি E-commerce Website তৈরি করতে হবে।

এখানে আপনি একটি Table-এ সব তথ্য রাখবেন না। বরং আলাদা আলাদা Table তৈরি করবেন।

- Customers
- Products
- Categories
- Orders
- Order Details
- Payments

এরপর Primary Key ও Foreign Key-এর মাধ্যমে Table-গুলোর মধ্যে সম্পর্ক তৈরি করবেন।

2. Database Infrastructure Design

Database Infrastructure Design কী?

Database Infrastructure Design হলো Database চালানোর জন্য প্রয়োজনীয় Hardware, Storage, Network, Operating System এবং High Availability Environment পরিকল্পনা করার প্রক্রিয়া।

অর্থাৎ Database শুধু Table Design নয়, Database কোথায় চলবে, কীভাবে চলবে এবং কত User Handle করবে—সেটিও Infrastructure Design-এর অংশ।

Database Infrastructure-এর প্রধান উপাদান

Server

Database যেখানে Install ও Run হবে।

CPU

Query Processing, Sorting, Index Scan এবং Parallel Execution পরিচালনা করে।

RAM

Buffer Cache, Shared Pool এবং Frequently Used Data Memory-তে রাখে, ফলে Disk I/O কমে।

Storage

Database Data File, Redo Log, Archive Log এবং Backup সংরক্ষণ করে।

Database Engineering for Developers & DBAs

Network

Application Server এবং Database Server-এর মধ্যে যোগাযোগ নিশ্চিত করে।

Backup Server

Backup File নিরাপদে সংরক্ষণ করে।

Disaster Recovery (DR) Site

Primary Database নষ্ট হলে DR Site থেকে Service চালু করা যায়।

Cloud Infrastructure

AWS, Azure, Google Cloud অথবা Oracle Cloud-এ Database Host করা যায়।

3. Database Architecture

Database Architecture কী?

Database Architecture বলতে Database System কীভাবে ডিজাইন করা হয়েছে এবং বিভিন্ন Component কীভাবে একে অপরের সাথে যোগাযোগ করে তা বোঝায়।

সহজভাবে বলতে গেলে, **Database Architecture** নির্ধারণ করে—

- User কীভাবে Database ব্যবহার করবে।
- Application কীভাবে Database-এর সাথে যোগাযোগ করবে।
- Database Server কীভাবে Data সংরক্ষণ ও পরিচালনা করবে।
- পুরো System কীভাবে নিরাপদ (Secure), দ্রুত (Fast) এবং নির্ভরযোগ্য (Reliable) হবে।

Database Architecture-এর উপর একটি System-এর **Performance, Scalability, Security, Availability এবং Maintenance** অনেকাংশে নির্ভর করে।

Database Architecture-এর প্রধান ধরন

Database Architecture সাধারণত তিনটি ভাগে বিভক্ত—

1. One-Tier Architecture
2. Two-Tier Architecture

Database Engineering for Developers & DBAs

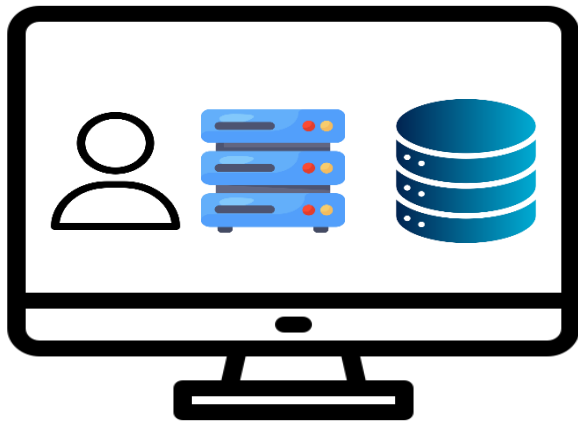
3. Three-Tier Architecture

১. One-Tier Architecture কী?

One-Tier Architecture হলো সবচেয়ে সহজ Database Architecture, যেখানে **User, Application এবং Database একই Machine-এ** থাকে।

অর্থাৎ, Database ব্যবহারকারী সরাসরি Database-এর সাথে কাজ করে।

Architecture



Client Application + Database Server (Same Computer)

কোথায় ব্যবহার হয়?

- Personal Project
- Desktop Application
- Database Learning
- Small Business Software

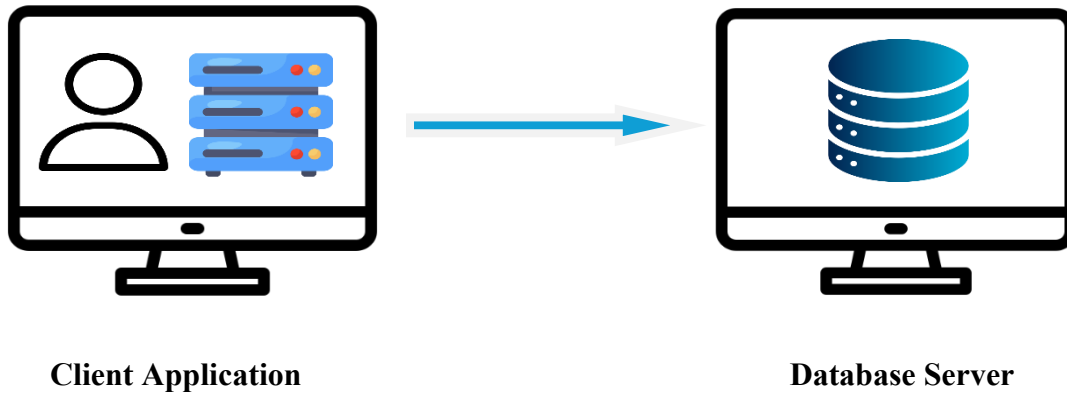
২. Two-Tier Architecture কী?

Two-Tier Architecture-এ **Client Application এবং Database Server আলাদা** থাকে।

Application সরাসরি Database Server-এর সাথে যোগাযোগ করে।

Architecture

Database Engineering for Developers & DBAs



কোথায় ব্যবহার হয়?

- Office Management System
- POS Software
- Inventory System
- Small ERP
- Internal Business Application

৩. Three-Tier Architecture কী?

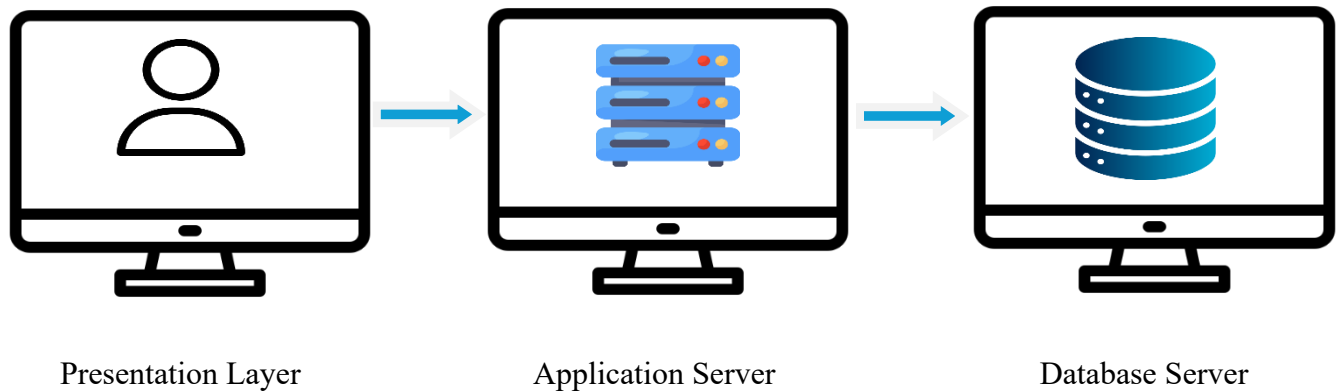
Three-Tier Architecture বর্তমানে সবচেয়ে জনপ্রিয় Architecture।

এখানে System তিনটি Layer-এ বিভক্ত থাকে—

- Presentation Layer
- Application Layer
- Database Layer

Database Engineering for Developers & DBAs

Architecture



প্রতিটি Layer-এর কাজ

১. Presentation Layer

এটি User Interface (UI), যেখানে User কাজ করে।

উদাহরণ:

- Web Browser
- Mobile App
- Desktop Application

২. Application Layer

এখানে Business Logic থাকে।

যেমন—

- Login Validation
- Payment Processing
- Order Processing
- Invoice Generation
- API Service

৩. Database Layer

এই Layer Data সংরক্ষণ ও পরিচালনা করে।

Database Engineering for Developers & DBAs

এখানে থাকে—

- Tables
- Indexes
- Views
- Stored Procedures
- Transactions

কীভাবে কাজ করে?

ধরুন একজন Customer একটি E-commerce Website থেকে Laptop কিনতে চান।

ধাপগুলো হবে—

1. Customer Website-এ Login করলেন।
2. Application Server Login যাচাই করল।
3. Product Search Request Database-এ পাঠানো হলো।
4. Database Product Information ফেরত দিল।
5. Customer Order Confirm করলেন।
6. Payment সম্পন্ন হলো।
7. Database Order, Payment এবং Inventory Update করল।
8. Application Confirmation Message দেখাল।

কোথায় ব্যবহার হয়?

- Banking System
- ERP
- Hospital Management System
- E-commerce Website
- Government System

Database Engineering for Developers & DBAs

- Airline Reservation
- University Management System

Architecture Comparison

বৈশিষ্ট্য	One-Tier	Two-Tier	Three-Tier
Layer সংখ্যা	1	2	3
Multi-user Support	সীমিত	ভালো	খুব ভালো
Security	কম	মাঝারি	উচ্চ
Scalability	নেই	মাঝারি	খুব ভালো
Maintenance	সহজ	মাঝারি	সহজ (Layer ভিত্তিক)
Performance	ছোট সিস্টেমে ভালো	মাঝারি	Enterprise পর্যায়ে সর্বোত্তম
ব্যবহার	Learning, Desktop App	Office Application	Enterprise Application

বাস্তব উদাহরণ

One-Tier

- SQLite ভিত্তিক Desktop Software
- ব্যক্তিগত Inventory Software

Two-Tier

- Oracle Forms Application
- POS Software
- Small ERP

Three-Tier

- Amazon
- Facebook
- Banking System

Database Engineering for Developers & DBAs

- Hospital Management System
- ERP (SAP, Oracle ERP)
- Government Portal

4. Database Modeling

Database Modeling কী?

Database Modeling হলো Database তৈরির আগে Data-এর Logical Structure ডিজাইন করার প্রক্রিয়া।

এতে নির্ধারণ করা হয়—

- কোন কোন Entity থাকবে।
- কী Attribute থাকবে।
- কীভাবে Entity-গুলোর মধ্যে Relationship থাকবে।

Database Modeling-এর ধাপ

Conceptual Model

Conceptual Model হলো Database Modeling-এর প্রথম ধাপ। এই ধাপে শুধুমাত্র **Business Requirement** বিশ্লেষণ করা হয় এবং কোন কোন Entity থাকবে ও তাদের মধ্যে কী ধরনের **Relationship** থাকবে তা নির্ধারণ করা হয়।

এখানে Table, Data Type, Primary Key বা Foreign Key নিয়ে চিন্তা করা হয় না।

সহজভাবে,

Conceptual Model = Business কী চায়, সেটি বোঝানোর Model।

উদাহরণ:

ধরুন একটি University-এর জন্য Database তৈরি করতে হবে।

Business Requirement অনুযায়ী নিচের Entity গুলো থাকবে।

- Student
- Teacher
- Course
- Department

Database Engineering for Developers & DBAs

- Enrollment

এগুলোই হলো **Conceptual Model-এর Entity**।

Logical Model

Logical Model হলো Database Modeling-এর দ্বিতীয় ধাপ। এই ধাপে Conceptual Model-কে বিস্তারিতভাবে Database Structure-এ রূপ দেওয়া হয়।

এখানে Entity থেকে **Table**, Attribute, Primary Key, Foreign Key এবং Relationship নির্ধারণ করা হয়।

তবে এখনো Data Type নির্ধারণ করা হয় না।

সহজভাবে,

Logical Model = Database-এর Structure Design।

উদাহরণ:

University Database (Logical Model)

STUDENT

Column
Student_ID (PK)
Student_Name
Department_ID (FK)

DEPARTMENT

Column
Department_ID (PK)
Department_Name

Database Engineering for Developers & DBAs

Physical Model

Physical Model হলো Database Modeling-এর শেষ ধাপ। এই ধাপে Logical Model-কে বাস্তব Database-এ (Oracle, PostgreSQL, MySQL, SQL Server ইত্যাদি) বাস্তবায়ন করা হয়।

এখানে Data Type, Constraint, Index, Sequence, Trigger, Partition, Storage ইত্যাদি নির্ধারণ করা হয়।

সহজভাবে,

Physical Model = Database Server-এ বাস্তব Implementation।

উদাহরণ:

```
CREATE TABLE STUDENT
(
    STUDENT_ID    NUMBER PRIMARY KEY,
    STUDENT_NAME  VARCHAR2(100) NOT NULL,
    DEPARTMENT_ID NUMBER,

    CONSTRAINT FK_STUDENT_DEPT
    FOREIGN KEY (DEPARTMENT_ID)
    REFERENCES DEPARTMENT(DEPARTMENT_ID)
);
```

এখানে দেখা যাচ্ছে—

- NUMBER এবং VARCHAR2 হলো Data Type।
- PRIMARY KEY তৈরি করা হয়েছে।
- FOREIGN KEY তৈরি করা হয়েছে।
- NOT NULL Constraint যোগ করা হয়েছে।

এগুলোই **Physical Model**-এর অংশ।

Database Engineering for Developers & DBAs

5. ER Diagram (Entity Relationship Diagram)

ER Diagram কী?

ER Diagram হলো Database Design-এর একটি Graphical Representation, যা Entity, Attribute এবং Relationship-কে চিত্রের মাধ্যমে উপস্থাপন করে।

Database তৈরি করার আগে ER Diagram তৈরি করা সবচেয়ে গুরুত্বপূর্ণ ধাপগুলোর একটি।

ER Diagram-এর সুবিধা

- Database Design সহজ হয়।
- Requirement বোঝা সহজ হয়।
- Development Team-এর Communication উন্নত হয়।
- Database Structure আগে থেকেই পরীক্ষার থাকে।

উদাহরণ

ধরুন একটি E-Commerce Website-এর Database তৈরি করতে হবে।

এখানে প্রধান Entity গুলো হলো—

- Customer
- Order
- Product

একজন Customer অনেক Order করতে পারেন এবং একটি Order-এর মধ্যে একাধিক Product থাকতে পারে।

ER Diagram



Database Engineering for Developers & DBAs

6. Entity

Entity কী?

Entity হলো এমন একটি বাস্তব Object বা বিষয়, যার সম্পর্কে Database-এ তথ্য সংরক্ষণ করা হয়।

সহজভাবে,

যার সম্পর্কে Data সংরক্ষণ করা হয়, সেটিই Entity।

উদাহরণ

একটি School Management System-এ—

- Student
- Teacher
- Course
- Department

সবগুলোই Entity।

আর একটি E-commerce System-এ—

- Customer
- Product
- Order
- Payment

সবগুলো Entity।

7. Attribute

Attribute কী?

Attribute হলো কোনো Entity-এর বৈশিষ্ট্য (Property) বা তথ্য।

অর্থাৎ Entity সম্পর্কে যেসব তথ্য সংরক্ষণ করা হয়, সেগুলোই Attribute।

Database Engineering for Developers & DBAs

উদাহরণ

Entity = Student

Attribute

- Student_ID
- Name
- Email
- Phone
- Date of Birth

Entity = Product

Attribute

- Product_ID
- Product_Name
- Price
- Stock
- Brand

Attribute-এর ধরন

Simple Attribute

আর ভাগ করা যায় না।

উদাহরণ

Age

Gender

Salary

Composite Attribute

একাধিক অংশে ভাগ করা যায়।

উদাহরণ

Database Engineering for Developers & DBAs

Address --- HouseNo, Road, City, Country

Single-Valued Attribute

একটি মাত্র Value থাকে।

উদাহরণ

Date of Birth

Multi-Valued Attribute

একাধিক Value থাকতে পারে।

উদাহরণ

Phone Number

Email Address

Derived Attribute

অন্য Attribute থেকে বের করা যায়।

উদাহরণ

Age (Date of Birth থেকে)

8. Relationship

Relationship কী?

Relationship হলো দুই বা ততোধিক Entity-এর মধ্যে সংযোগ (Association)।

উদাহরণ

Customer একটি Order দেয়।

অর্থাৎ

Customer ---- Places ---- Order

Database Engineering for Developers & DBAs

Student একটি Course-এ ভর্তি হয়।

Student ---- Enrolls ---- Course

Doctor রোগীকে চিকিৎসা করেন।

Doctor ---- Treats ---- Patient

9. Cardinality

Cardinality কী?

Cardinality বোঝায় একটি Entity-এর কতগুলো Record অন্য Entity-এর কতগুলো Record-এর সাথে সম্পর্কিত হতে পারে।

সহজভাবে,

একটি Table-এর একটি Record অন্য Table-এর কতটি Record-এর সাথে সম্পর্ক তৈরি করতে পারে, সেটিই Cardinality।

1. One-to-One (1:1)

One-to-One (1:1) Relationship হলো এমন একটি Relationship যেখানে একটি Table-এর একটি Record-এর সাথে অন্য Table-এর শুধুমাত্র একটি Record সম্পর্কিত থাকে।

সহজভাবে বলা যায়,

One Parent → One Child

উদাহরণ: Person এবং Passport

একজন ব্যক্তির একটি মাত্র Passport থাকে এবং একটি Passport শুধুমাত্র একজন ব্যক্তিরই হয়।

Person Table (Parent)

Person_ID (PK)	Name	Date_of_Birth
101	Rahim	15-Jan-1995
102	Karim	20-Feb-1996

Database Engineering for Developers & DBAs

Passport Table (Child)

Passport_ID (PK)	Passport_No	Person_ID (FK - UNIQUE)
P001	BD1234567	101
P002	BD9876543	102

এখানে,

- **Rahim (Person_ID = 101)**-এর একটি Passport (**P001**) রয়েছে।
- **Karim (Person_ID = 102)**-এর একটি Passport (**P002**) রয়েছে।
- একটি Person-এর একাধিক Passport নেই।
- একটি Passport একাধিক Person-এর হতে পারে না।

2. One-to-Many (1:N)

One-to-Many (1:N) হলো Database-এর সবচেয়ে বেশি ব্যবহৃত Relationship। এখানে একটি Parent Record-এর সাথে একাধিক Child Record সম্পর্কিত থাকতে পারে, কিন্তু একটি Child Record শুধুমাত্র একটি Parent Record-এর সাথেই সম্পর্কিত থাকবে।

সহজভাবে বলা যায়,

One Parent → Many Child

উদাহরণ ১: Customer এবং Order

একজন Customer অনেকগুলো Order করতে পারেন, কিন্তু একটি Order শুধুমাত্র একজন Customer-এরই হবে।

Customer Table (Parent)

Customer_ID (PK)	Customer_Name	Phone
1001	Rahim	017XXXXXXXXX
1002	Karim	018XXXXXXXXX

Database Engineering for Developers & DBAs

Order Table (Child)

Order_ID (PK)	Customer_ID (FK)	Product	Amount
5001	1001	Laptop	80000
5002	1001	Mouse	700
5003	1001	Keyboard	1500
5004	1002	Monitor	25000

এখানে,

- **Rahim (Customer_ID = 1001) মোট ৩টি Order করেছেন।**
- **Karim (Customer_ID = 1002) ১টি Order করেছেন।**

3. Many-to-One (N:1)

যখন একটি টেবিলের একাধিক রেকর্ড (Many) অন্য একটি টেবিলের একটি মাত্র রেকর্ডের (One) সাথে যুক্ত থাকে, তখন তাকে Many-to-One রিলেশনশিপ বলে।

উদাহরণ: একটি অফিসে অনেক কর্মচারী (Employee) থাকতে পারে, কিন্তু তাদের মধ্যে কয়েকজন মিলে একটি নির্দিষ্ট বিভাগ বা ডিপার্টমেন্টের (Department) অধীনে কাজ করে।

এই রিলেশনশিপটি বাস্তবে কীভাবে কাজ করে, তা বোঝানোর জন্য নিচে দুটি টেবিলের গঠন দেওয়া হলো:

১. Departments Table (ডিপার্টমেন্টের মূল টেবিল - 'One' সাইড)

এখানে শুধু ডিপার্টমেন্টগুলোর তথ্য থাকে। DeptID হলো এই টেবিলের প্রাইমারি কি (Primary Key)।

DeptID (PK)	DepartmentName	Location
D-10	HR (Human Resources)	2nd Floor
D-20	IT (Information Technology)	3rd Floor
D-30	Finance	4th Floor

Database Engineering for Developers & DBAs

২. Employees Table (কর্মচারীদের মূল টেবিল - 'Many' সাইড)

এখানে কর্মচারীদের নিজস্ব তথ্য থাকে। এই টেবিলে কোন কর্মচারী কোন ডিপার্টমেন্টে কাজ করে তা বোঝানোর জন্য DeptID-কে ফরেন কি (Foreign Key) হিসেবে ব্যবহার করা হয়েছে, যা ডিপার্টমেন্ট টেবিল থেকে এসেছে।

EmpID (PK)	EmployeeName	JobTitle	DeptID (FK)
1001	Rahim	HR Manager	D-10
1002	Karim	Software Engineer	D-20
1003	Sumit	Database Admin	D-20
1004	Anik	Data Analyst	D-20

এই টেবিলগুলো থেকে আমরা কী বুঝতে পারছি?

- **Karim, Sumit এবং Anik:** এই তিনজন আলাদা কর্মচারী (Many), কিন্তু তারা সবাই একটি মাত্র ডিপার্টমেন্ট IT (D-20)-এর অধীনে কাজ করছে।
- **Rahim:** সে HR (D-10) ডিপার্টমেন্টে কাজ করছে।
- অর্থাৎ, একাধিক Employee-এর ডেটা একটি মাত্র Department-এর ডেটার সাথে যুক্ত হয়ে Many-to-One সম্পর্ক তৈরি করেছে।

4. Many-to-Many (M:N)

অনেক Student অনেক Course করতে পারে।

একটি Course-এ অনেক Student থাকতে পারে।

এক্ষেত্রে সাধারণত একটি **Junction Table** ব্যবহার করা হয়।

উদাহরণ :

ধরে নিন আমাদের একটি স্টুডেন্ট টেবিল এবং একটি কোর্স টেবিল আছে। যেহেতু একজন স্টুডেন্ট অনেক কোর্স নিতে পারে এবং একটি কোর্সে অনেক স্টুডেন্ট থাকতে পারে, তাই এদের মাঝে সম্পর্ক তৈরির জন্য একটি তৃতীয় টেবিল (Junction Table) লাগবে।

নিচে তিনটি টেবিলের গঠন দেওয়া হলো:

Database Engineering for Developers & DBAs

১. Students Table (ছাত্রদের মূল টেবিল)

এখানে প্রতিটি ছাত্রের নিজস্ব তথ্য থাকে। StudentID হলো এই টেবিলের প্রাইমারি কি (Primary Key)।

StudentID (PK)	StudentName	Age	Department
101	Rahim	21	CSE
102	Karim	22	EEE
103	Sumit	20	BBA

২. Courses Table (কোর্সের মূল টেবিল)

এখানে কোর্সগুলোর তথ্য থাকে। CourseID হলো এই টেবিলের প্রাইমারি কি (Primary Key)।

CourseID (PK)	CourseName	Credits
C-01	Database Systems	3
C-02	Python Programming	3
C-03	Basic Accounting	2

৩. Enrollments Table (Junction Table / সংযোগকারী টেবিল)

এই টেবিলটি উপরের দুটি টেবিলকে যুক্ত করেছে। এখানে StudentID এবং CourseID হলো ফরেন কি (Foreign Key), যা মূল টেবিলগুলো থেকে এসেছে।

EnrollmentID (PK)	StudentID (FK)	CourseID (FK)	EnrollmentDate
1	101	C-01	2026-07-25

Database Engineering for Developers & DBAs

EnrollmentID (PK)	StudentID (FK)	CourseID (FK)	EnrollmentDate
2	101	C-02	2026-07-25
3	102	C-01	2026-07-26
4	103	C-03	2026-07-28

Database Design Best Practices

- Business Requirement আগে ভালোভাবে বুঝুন।
- সঠিক Data Type নির্বাচন করুন।
- Primary Key ও Foreign Key ব্যবহার করুন।
- প্রয়োজন অনুযায়ী Normalization করুন।
- Performance-এর জন্য Index ব্যবহার করুন।
- Security ও Access Control নিশ্চিত করুন।
- Backup ও Recovery পরিকল্পনা রাখুন।
- Scalability মাথায় রেখে Design করুন।
- Documentation ও ER Diagram সংরক্ষণ করুন।

Chapter Summary

এই অধ্যায়ে আমরা শিখেছি—

- Database Design কী এবং কেন গুরুত্বপূর্ণ।
- Database Infrastructure Design-এর মূল উপাদান।
- Database Architecture এবং Three-Tier Architecture।
- Database Modeling-এর ধাপ।
- ER Diagram-এর ভূমিকা।
- Entity, Attribute ও Relationship-এর ধারণা।
- Cardinality-এর বিভিন্ন ধরন এবং বাস্তব উদাহরণ।